

HiHome

Applicazioni e Servizi Web

Denise Nanni - 0001176519 {denise.nanni@studio.unibo.it}
Juri Guglielmi - 0001182994 {juri.guglielmi@studio.unibo.it}
Nicolò Monaldini - 0001186789 {nicolo.monaldini@studio.unibo.it}

Giugno 2026

Indice

1	Introduzione	3
2	Requisiti	4
2.1	Requisiti funzionali	4
2.2	Requisiti non funzionali	5
2.3	Requisiti di implementazione	5
3	Design	7
3.1	User Interface	7
3.2	Design Architettuale	13
3.3	Progettazione API	14
3.4	Analisi degli Utenti Target	16
3.5	Definizione delle Personas e Scenari d'Uso	16
3.5.1	Persona 1: Marco, il padre di famiglia	17
3.5.2	Persona 2: Giulia, la ragazza tecnologica	17
3.5.3	Persona 3: Elena, la madre attenta al benessere	17
4	Tecnologie	18
4.1	Backend	18
4.2	Frontend	18
4.3	Database	18
4.4	Infrastruttura	18
5	Codice	20
5.1	Logica di Autorizzazione	20
5.2	Validazione Input tramite Middleware	20
5.3	Notifiche Real-Time con Socket.io	21
5.4	Integrazione con Modello AI (Streaming)	21

6	Test	23
6.1	Euristiche di Nielsen	23
6.2	Unit Testing e Strumenti di sviluppo	24
6.3	Test di usabilità	24
6.3.1	Metriche	24
6.3.2	Elenco dei task	24
6.3.3	Dati rilevati	25
7	Deployment	27
7.1	Comandi per il Deployment	27
7.1.1	Esecuzione Container (Produzione)	27
7.1.2	Esecuzione con Hot Reloading	27
7.1.3	Esecuzione dei Test	27
7.2	Endpoint e Indirizzi Locali	28
8	Conclusioni	29
9	Screenshots	30

1 Introduzione

Il progetto HiHome consiste in un applicativo per gestire dispositivi all'interno di una smart home. Permette di gestire dispositivi smart da applicazione e aggiungere automazioni. Sono presenti funzionalità come notifiche, log di eventi relativi ai device smart, informazioni sullo stato attuale di temperatura, meteo e qualità dell'aria nella zona e grafici relativi ai valori passati di questi ultimi. Inoltre, è integrato con un agente AI in modo da effettuare alcune azioni a partire da istruzioni in linguaggio naturale. L'applicazione supporta due tipi di utenti:

- **Admin:** ha accesso a funzioni avanzate sulla gestione della casa, come log di eventi relativi a manipolazione di smart device, creazione di automazioni e aggiunta di device.
- **Utente Standard:** può agire sugli smart device, visualizzare statistiche relative allo stato attuale della casa, grafici relativi ad eventi passati, come lo stato della qualità dell'aria nella zona durante la settimana precedente e interagire con il chatbot.

2 Requisiti

2.1 Requisiti funzionali

- Login
- Menu per navigazione tra le pagine
- Dashboard con informazioni sullo stato attuale della casa e del circondario:
 - Temperatura (interna ed esterna)
 - Condizioni meteo attuali
 - Qualità dell'aria
 - Velocità e direzione del vento
- Lista dispositivi smart, divisi per stanza, con possibilità di interagire con essi; in particolare:
 - Accensione/spegnimento per luci
 - Apertura/chiusura per finestre
 - Impostazione temperatura per termostato
 - Apertura o chiusura per serrature
 - Controllo velocità di ventilatori
- (Admin) Possibilità di aggiunta di nuovi dispositivi smart, con specifica del tipo di dispositivo, del nome e della stanza in cui si trova
- (Admin) Possibilità di creare automazioni, ognuna definita da:
 - Nome
 - Condizione di attivazione composta da un sensore, un operatore e un valore
 - Finestra temporale opzionale che permette di specificare in quali giorni e orari l'automazione deve essere attiva
 - Una o più azioni da eseguire quando la condizione è verificata, ognuna composta da un dispositivo smart e un'azione da eseguire su di esso
- (Admin) Possibilità di visualizzare la lista completa di automazioni presenti
- (Admin) Possibilità di visualizzare un log di eventi relativi alla manipolazione dei dispositivi smart, con specificate data, ora, dispositivo interessato, azione eseguita e utente che ha eseguito l'azione
- Grafici relativi a dati passati, in particolare:
 - Grafico della temperatura esterna durante la settimana precedente, con temperatura massima e minima per ogni giorno
 - Andamento della qualità dell'aria durante la settimana precedente
 - Grafico di utilizzo dei dispositivi smart durante la settimana precedente, con numero di azioni per fascia oraria

- Dati relativi a numero di attivazioni manuali e automatiche per i dispositivi smart
- Dati relativi alla percentuale di tempo settimanale in cui le luci sono rimaste accese e le finestre aperte
- Chatbot con cui è possibile interagire in linguaggio naturale per eseguire azioni e chiedere informazioni a seconda dei privilegi dell'utente, ovvero:
 - Aggiungere un dispositivo smart (Admin)
 - Aggiungere una regola di automazione (Admin)
 - Chiedere informazioni relative alle previsioni meteo per la settimana successiva
 - Domande generiche limitatamente all'ambito delle smart home
- Notifiche relative a:
 - Esecuzione di un'automazione
 - Livelli critici della qualità dell'aria
 - (Admin) Azioni eseguite da utenti non admin
- Impostazioni, con possibilità di:
 - Effettuare logout
 - (Admin) Gestire gli utenti, con possibilità di visualizzare e modificare i permessi degli utenti della casa
 - Definire quali notifiche ricevere
- Piano giornaliero delle temperature, con possibilità di visualizzare la temperatura prevista per ogni ora del giorno e di impostare una temperatura desiderata per ogni fascia oraria

2.2 Requisiti non funzionali

- Aggiornamento in tempo reale dello stato dei dispositivi smart
- Interfaccia user-friendly e responsive
- Utilizzo di token per autenticazione della sessione
- Aggiornamento della chat con streaming delle risposte da parte del modello AI, per evitare tempi di attesa eccessivi

2.3 Requisiti di implementazione

- Utilizzo di API RESTful per comunicazione tra frontend e backend
- Architettura MEVN (MongoDB, Express.js, Vue.js, Node.js), con aggiunta di un servizio esterno in Go per ottenere dati meteo e qualità dell'aria
- Utilizzo di Socket.io per notifiche, aggiornamenti in tempo reale e streaming delle risposte del modello AI

- Applicazione web responsive e mobile-first, con l'utilizzo di Tailwind CSS
- Utilizzo di Docker come tecnologia di containerizzazione per facilitare lo sviluppo e la distribuzione dell'applicazione
- Utilizzo di Semantic Versioning per il versionamento
- Git e Semantic Commit per la gestione del codice sorgente

3 Design

3.1 User Interface

L'interfaccia utente è progettata per essere intuitiva e facile da navigare, con un design moderno e pulito. Il design si è basato su un approccio mobile-first. L'usabilità su dispositivi mobile è stata quindi una priorità, e le interfacce grafiche sono state poi estese a schermi desktop tramite layout responsive. Per prima cosa, è stata realizzata una moodboard per identificare la palette colori da utilizzare e il tono generale dell'applicazione, visibile in Figura 1. Di seguito sono riportati i mockup delle principali schermate

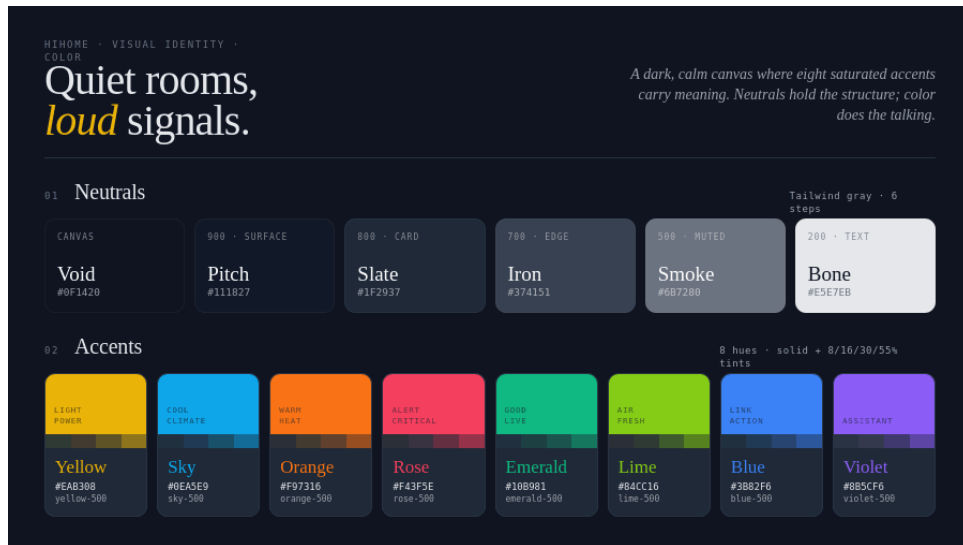
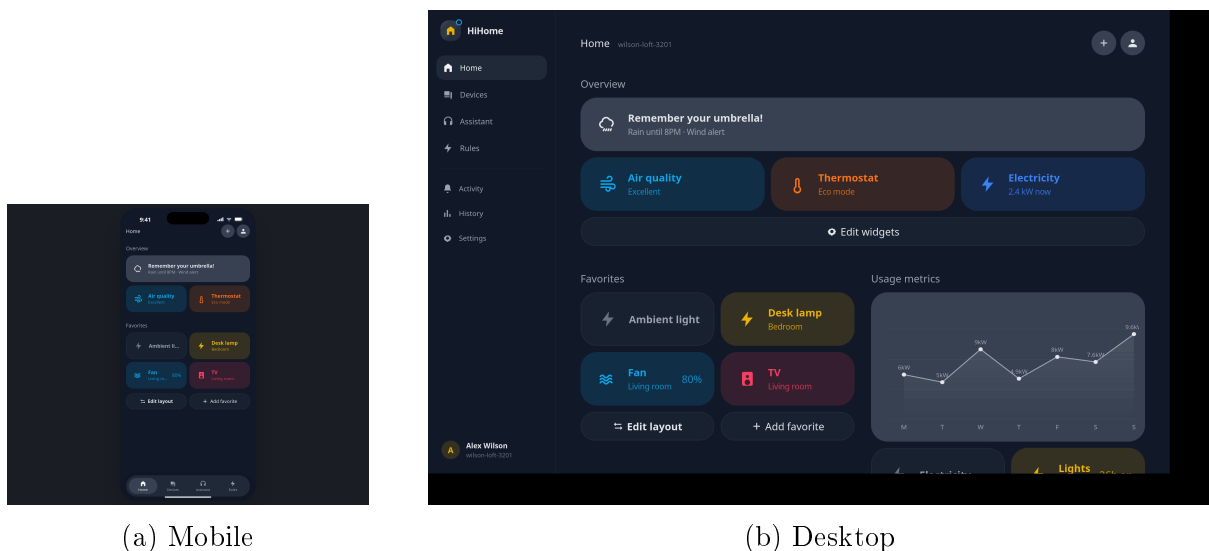


Figura 1: Moodboard per il design dell'applicazione

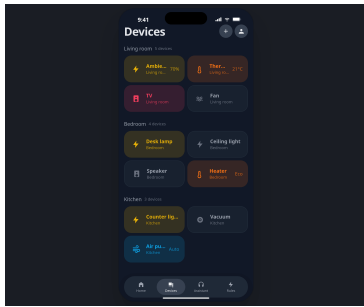
dell'applicazione, sia per la versione mobile che per la versione desktop. Sebbene siano state apportate lievi modifiche durante la fase di implementazione, l'interfaccia grafica è rimasta sostanzialmente fedele a questi mockup.



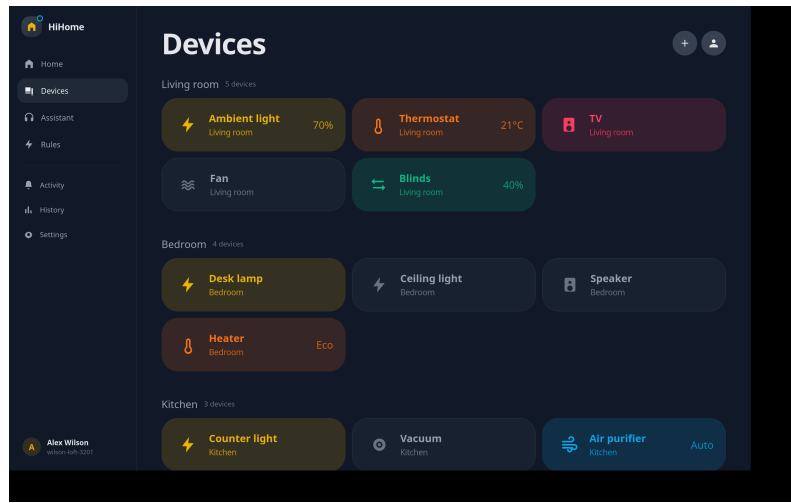
(a) Mobile

(b) Desktop

Figura 2: Dashboard

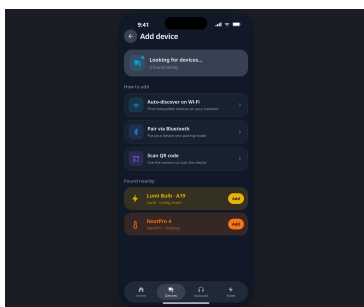


(a) Mobile

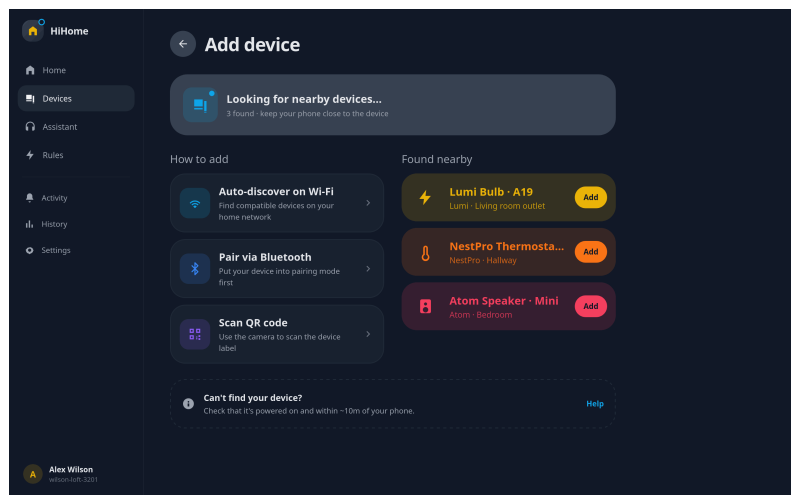


(b) Desktop

Figura 3: Lista device

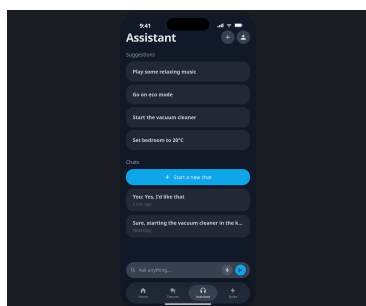


(a) Mobile

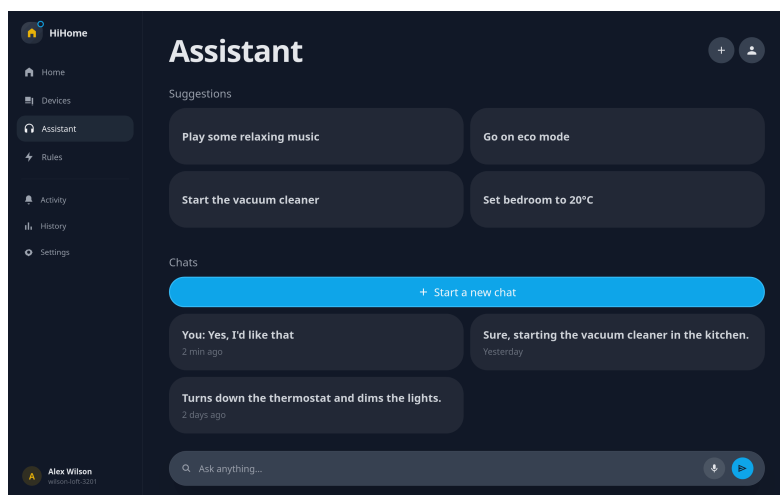


(b) Desktop

Figura 4: Aggiunta di un nuovo device

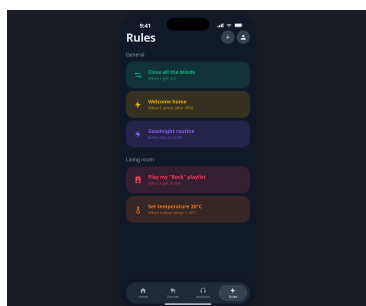


(a) Mobile

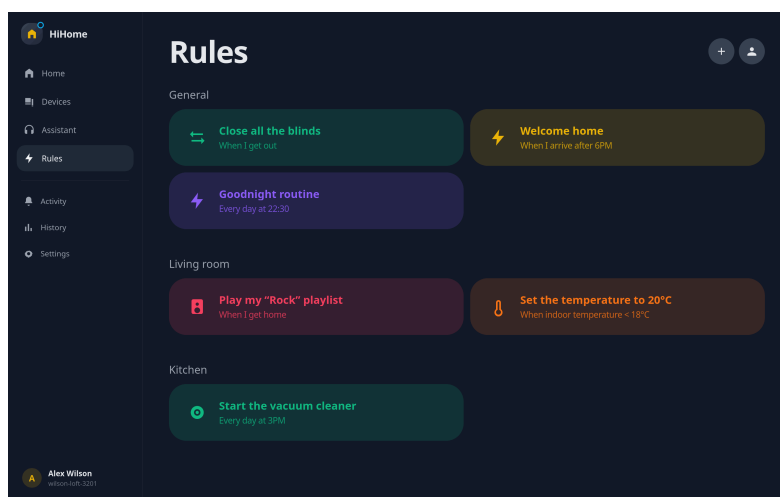


(b) Desktop

Figura 5: Chatbot

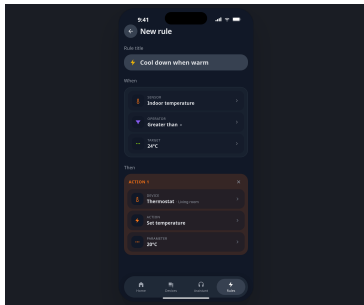


(a) Mobile

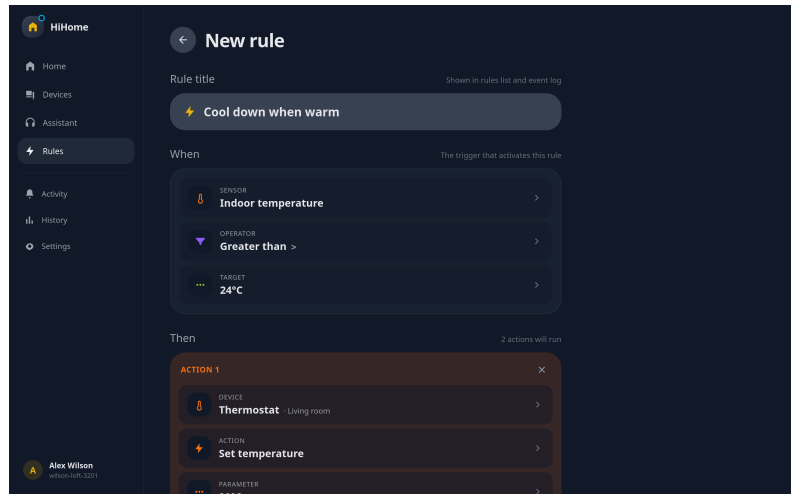


(b) Desktop

Figura 6: Regole di automazione

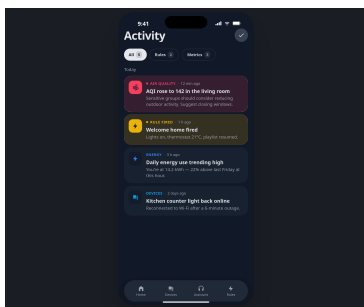


(a) Mobile

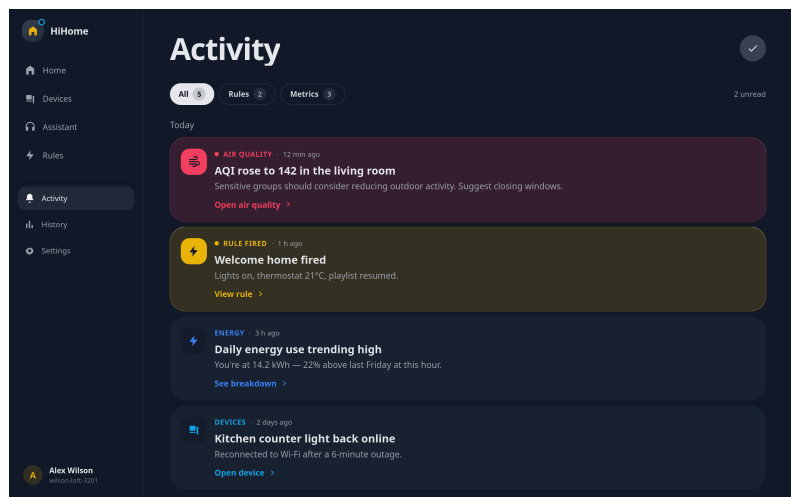


(b) Desktop

Figura 7: Creazione nuova regola di automazione

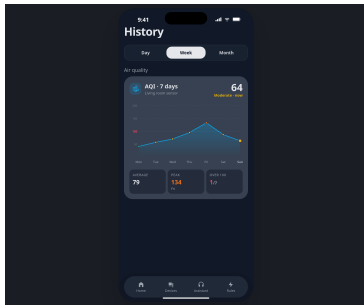


(a) Mobile

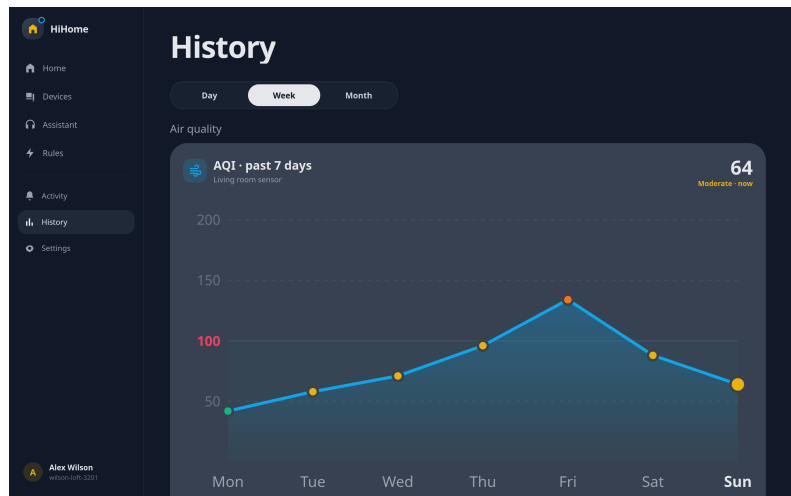


(b) Desktop

Figura 8: Notifiche

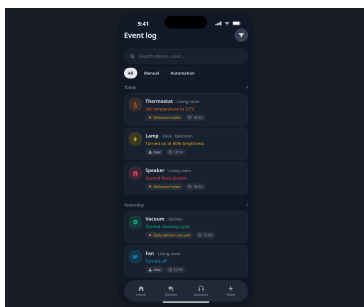


(a) Mobile

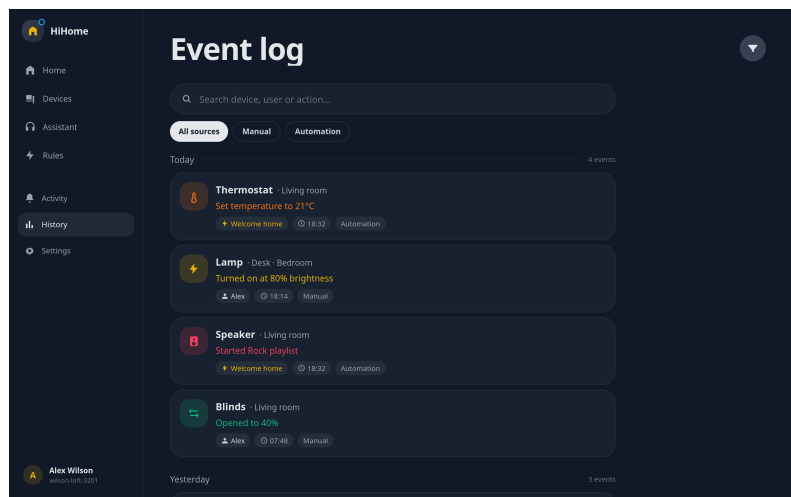


(b) Desktop

Figura 9: Report settimanale

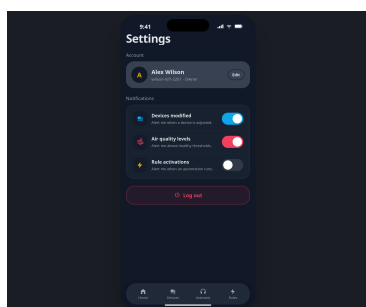


(a) Mobile

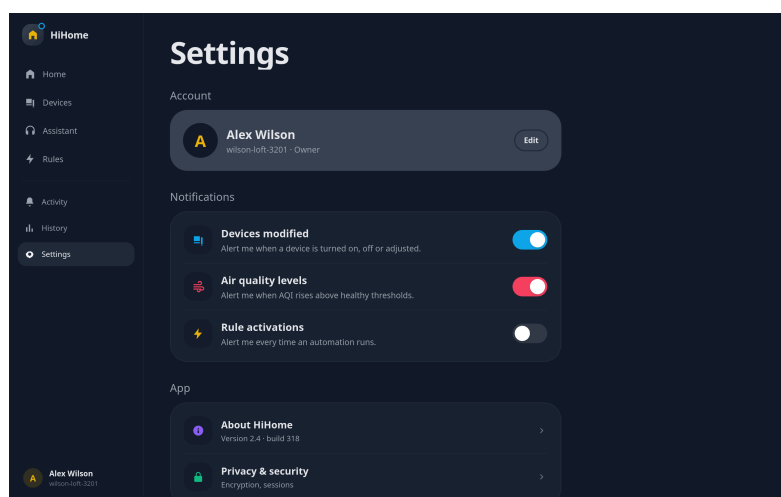


(b) Desktop

Figura 10: Log di eventi



(a) Mobile



(b) Desktop

Figura 11: Impostazioni

3.2 Design Architetturale

Il progetto è stato realizzato seguendo la filosofia di design **Domain-Driven Design (DDD)**. Come pattern architetturale, è stata adottata un'architettura esagonale (ports and adapters) per ogni bounded context identificato. In particolare, sono stati identificati i seguenti bounded contexts:

- **Home context:** è il contesto principale dell'applicazione, che si occupa della gestione delle case, dei dispositivi smart e della chat
- **User context:** si occupa della gestione degli utenti e dell'autenticazione
- **Notification context:** si occupa della gestione delle notifiche
- **Rule context:** si occupa della gestione delle regole di automazione
- **Weather context:** si occupa di ottenere dati relativi al meteo e alla qualità dell'aria da servizi esterni

L'architettura utilizza lo stack MEVN, con i primi 4 contesti implementati nel backend in Node.js, mentre il Weather context è implementato in un servizio esterno in Go.

L'applicazione HiHome è quindi divisa in diversi componenti.

- I **client**, che accedono all'applicazione tramite interfaccia web
- Il **backend**, che espone API RESTful per la comunicazione con i client e gestisce la business logic
- Il **servizio esterno per il meteo**, che è interrogato periodicamente dal backend per ottenere dati aggiornati relativi alle condizioni nelle zone in cui le varie case si trovano
- Il **Modello AI**, ovvero Deepseek-V4, al quale l'accesso avviene tramite API key

Il server backend, inoltre, accetta connessioni WebSocket dai client per l'invio di notifiche, l'invio aggiornamenti in tempo reale e per lo streaming delle risposte del modello AI. Il backend accede ai dati tramite un database MongoDB.

Per quanto riguarda i concetti di dominio, si definiscono di seguito i principali, già brevemente descritti in precedenza:

- **Casa:** rappresenta una casa intelligente, con i suoi dispositivi smart, le sue automazioni e i suoi utenti
- **Dispositivo Smart:** rappresenta un dispositivo intelligente all'interno di una casa, come una luce, una finestra o un termostato
- **Automazione:** rappresenta una regola di automazione, composta da una condizione di attivazione e da una o più azioni da eseguire quando la condizione è verificata
- **Utente:** rappresenta un utente dell'applicazione, con i suoi permessi e le sue preferenze

3.3 Progettazione API

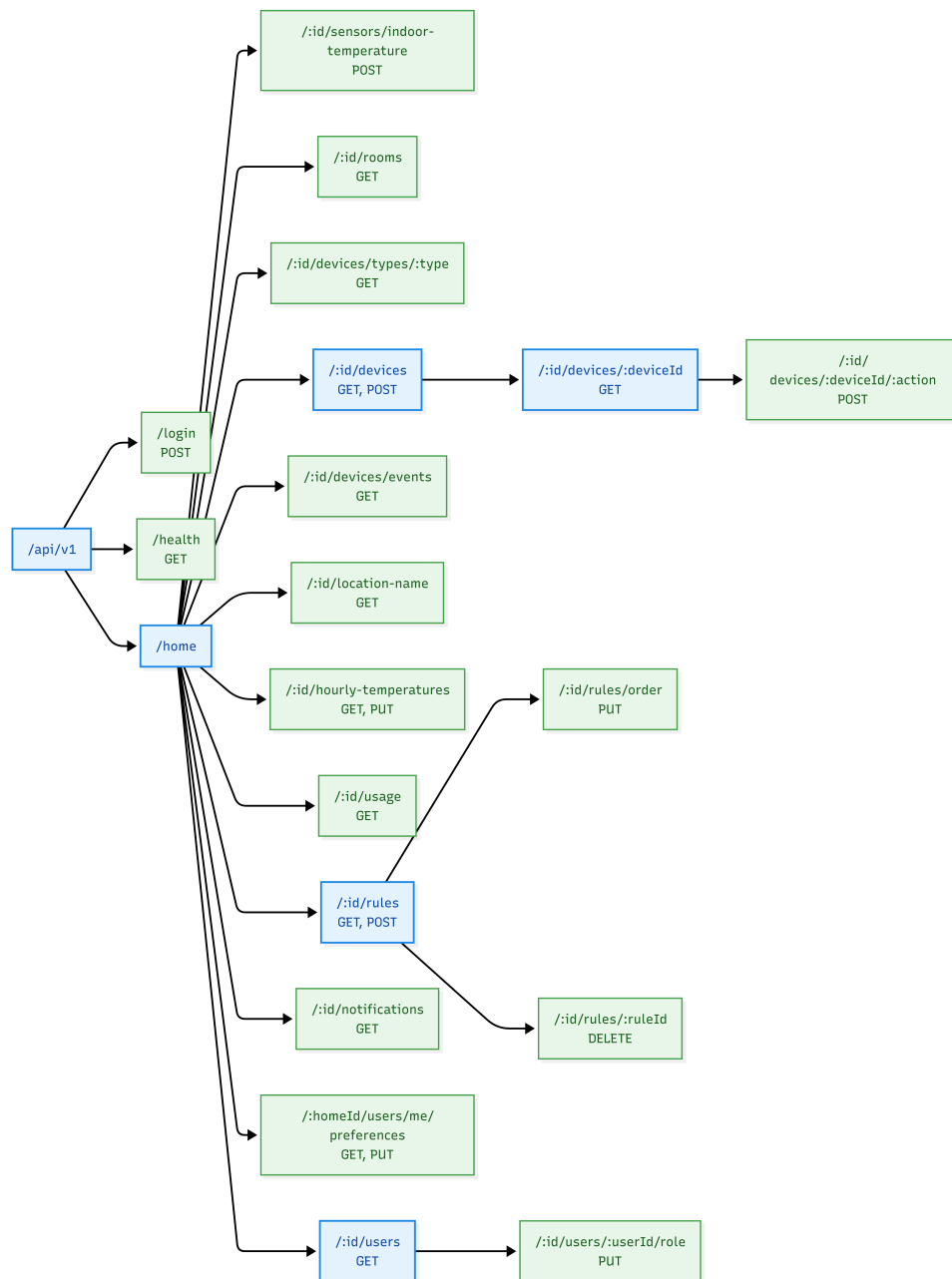


Figura 12: API Backend Node.js

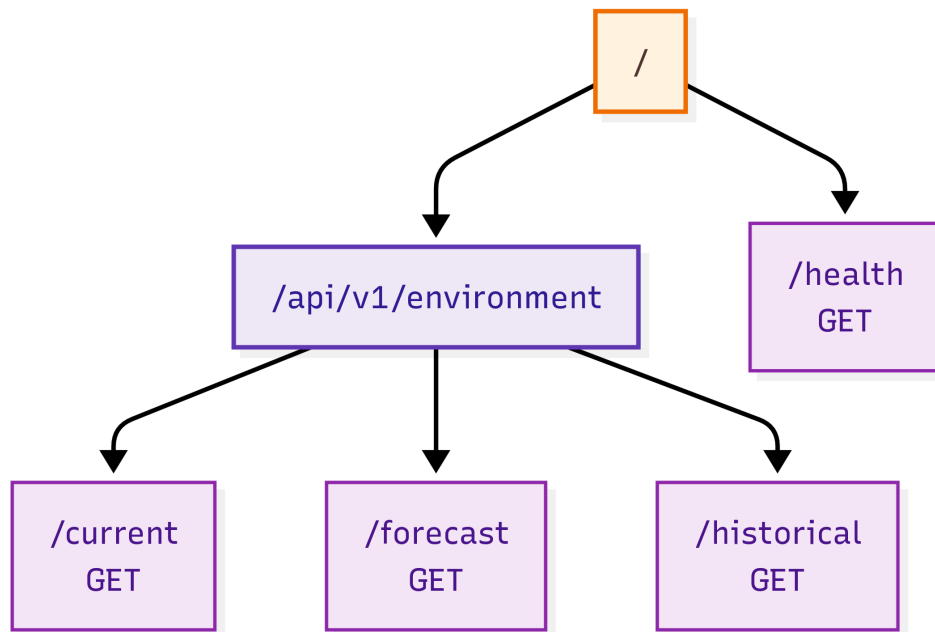


Figura 13: API Servizio Esterno Go (Meteo e Qualità dell'Aria)

In Figura 12 sono riportati i principali endpoint esposti dal backend Node.js, mentre in Figura 13 sono riportati i principali endpoint esposti dal servizio esterno Go per il meteo. Di seguito è riportata una breve descrizione di ogni endpoint:

- **Backend Node.js:**

- POST `/api/v1/login`: Autenticazione dell'utente e generazione del token di sessione.
- GET `/api/v1/health`: Endpoint di health check per verificare lo stato di salute del backend.
- POST `/api/v1/home/:id/sensors/indoor-temperature`: Aggiornamento della rilevazione della temperatura interna della casa.
- GET `/api/v1/home/:id/rooms`: Recupero della lista delle stanze presenti nella casa.
- GET `/api/v1/home/:id/devices/types/:type`: Ottenimento dei dispositivi smart filtrati per tipologia.
- GET/POST `/api/v1/home/:id/devices`: Ottenimento della lista dei dispositivi smart della casa o aggiunta di un nuovo dispositivo.
- GET `/api/v1/home/:id/devices/:deviceId`: Ottenimento dei dettagli di un dispositivo specifico.
- POST `/api/v1/home/:id/devices/:deviceId/:action`: Esecuzione di un'azione (es. accensione/spegnimento) su uno specifico dispositivo smart.
- GET `/api/v1/home/:id/devices/events`: Ottenimento del log degli eventi relativi ai dispositivi smart.
- GET `/api/v1/home/:id/location-name`: Recupero del nome della località in cui si trova la casa.

- GET/PUT /api/v1/home/:id/hourly-temperatures: Visualizzazione o modifica del piano giornaliero delle temperature.
- GET /api/v1/home/:id/usage: Recupero delle statistiche di utilizzo dei dispositivi.
- GET/POST /api/v1/home/:id/rules: Ottenimento della lista delle regole di automazione o creazione di una nuova regola.
- PUT /api/v1/home/:id/rules/order: Aggiornamento dell'ordine di valutazione delle automazioni.
- DELETE /api/v1/home/:id/rules/:ruleId: Eliminazione di una regola di automazione.
- GET /api/v1/home/:id/notifications: Ottenimento delle notifiche relative alla casa.
- GET/PUT /api/v1/home/:homeId/users/me/preferences: Visualizzazione o aggiornamento delle preferenze (es. ricezione notifiche) dell'utente corrente.
- GET /api/v1/home/:id/users: Ottenimento della lista degli utenti associati alla casa.
- PUT /api/v1/home/:id/users/:userId/role: Modifica dei privilegi (ruolo) di un utente specifico.

- **Servizio Esterno Go (Meteo e Qualità dell'Aria):**

- GET /health: Endpoint di health check per verificare lo stato del servizio.
- GET /api/v1/environment/current: Ottenimento delle condizioni meteorologiche e della qualità dell'aria attuali.
- GET /api/v1/environment/forecast: Ottenimento delle previsioni meteorologiche e della qualità dell'aria.
- GET /api/v1/environment/historical: Ottenimento dei dati storici su meteo e qualità dell'aria per i grafici.

3.4 Analisi degli Utenti Target

La fase di progettazione è stata preceduta da un'analisi del target di riferimento, volta a identificare le diverse tipologie di attori che interagiscono con l'applicazione e i loro requisiti.

Per prima cosa, è stata fatta la definizione di specifiche Personas. Ogni profilo rappresenta un utente caratterizzato da obiettivi e competenze tecnologiche peculiari, nonché dal ruolo che ricoprono all'interno dell'abitazione (Admin o Utente Standard).

Ogni Persona è associata a use case che descrivono la loro interazione con il sistema. Questi use case sono stati poi utilizzati come riferimento durante la fase di progettazione dell'interfaccia utente e delle funzionalità dell'applicazione, al fine di garantire che l'esperienza utente fosse adeguata alle esigenze specifiche di ogni tipologia di utente.

3.5 Definizione delle Personas e Scenari d'Uso

Di seguito vengono presentati i profili utente identificati e le situazioni tipiche in cui si trovano a interagire con il sistema.

3.5.1 Persona 1: Marco, il padre di famiglia

Profilo: 45 anni, impiegato. Ricopre il ruolo di amministratore della smart home.

Obiettivo: Gestire la casa in modo efficiente, ottimizzando i consumi energetici e garantendo il benessere della famiglia.

Scenario 1: Configurazione e automazioni. In vista dell'estate, Marco decide di ottimizzare la gestione della temperatura all'interno della casa. Accede alla sezione delle automazioni e crea una nuova regola: se la temperatura esterna sale sopra i 30 gradi, le finestre devono essere aperte.

3.5.2 Persona 2: Giulia, la ragazza tecnologica

Profilo: 21 anni, studentessa universitaria, esperta di tecnologia. Si è da poco trasferita.

Obiettivo: Aggiungere dispositivi alla smart home.

Scenario 2: Controllo dei dispositivi. Giulia ha acquistato diversi dispositivi smart e vuole aggiungerli alla sua casa. Accede alla sezione di gestione dei dispositivi, seleziona "Aggiungi dispositivo" e segue la procedura guidata per configurare ogni nuovo dispositivo, assegnandolo alla stanza corretta. Una volta aggiunti, interagisce con essi tramite app, accendendo le luci, regolando il termostato e chiudendo le finestre direttamente dal suo smartphone.

3.5.3 Persona 3: Elena, la madre attenta al benessere

Profilo: 42 anni, insegnante, utilizza lo smartphone ma non è un'esperta di domotica.

Obiettivo: Monitorare il benessere abitativo e tenere sotto controllo l'utilizzo della casa.

Scenario 3: Monitoraggio ambientale e notifiche. Elena soffre di allergie ed è particolarmente attenta alla qualità dell'aria. Ha configurato l'app per ricevere notifiche push nel caso in cui i livelli di inquinanti esterni superino la soglia di guardia. Ricevuta una notifica di allerta inquinamento, decide di chiudere le finestre di casa tramite l'app per evitare che l'aria esterna comprometta l'ambiente domestico. Consulta inoltre i grafici della qualità dell'aria per capire l'andamento della settimana.

4 Tecnologie

Il sistema è stato sviluppato utilizzando lo stack MEVN (MongoDB, Express.js, Vue.js, Node.js), con l'aggiunta di un servizio esterno in Go per ottenere dati meteo e sulla qualità dell'aria. Sono di seguito descritte le principali tecnologie e librerie principali utilizzate.

4.1 Backend

- **Typescript**: utilizzato in alternativa a JavaScript per lo sviluppo del backend, in quanto offre tipizzazione statica e una migliore manutenibilità del codice.
- **Express.js**: framework per Node.js utilizzato per la realizzazione del server backend, che espone API RESTful per la comunicazione con i client.
- **Socket.io**: libreria per la comunicazione in tempo reale tra client e server, utilizzata per l'invio di notifiche, aggiornamenti in tempo reale e streaming delle risposte del modello AI.
- **express-validator**: libreria per la convalida degli input degli utenti all'interno di Middleware, utilizzata per garantire la qualità e la sicurezza dei dati inseriti.
- **Vitest**: framework per i test unitari, utilizzato per assicurare il corretto funzionamento delle funzionalità principali.
- **Prettier**: strumento di formattazione del codice, utilizzato per mantenere uno stile di codice coerente e leggibile.

4.2 Frontend

- **Vue 3**: framework JavaScript utilizzato per la realizzazione dell'interfaccia utente, con un'architettura basata su componenti.
- **Tailwind CSS**: framework CSS utilizzato per lo styling dell'applicazione, che consente di creare interfacce responsive e moderne in modo efficiente.

4.3 Database

- **MongoDB**: database NoSQL utilizzato per la memorizzazione dei dati dell'applicazione, come informazioni sulle case, dispositivi smart, automazioni e utenti.
- **Mongoose**: libreria per Node.js che fornisce un'interfaccia di modellazione degli oggetti per MongoDB, semplificando l'interazione con il database e la definizione degli schemi dei dati.

4.4 Infrastruttura

- **Nginx**: utilizzato come server web per servire l'applicazione frontend e come reverse proxy per il backend.
- **Docker**: utilizzato per la containerizzazione dell'applicazione (backend, frontend, servizio API meteo), facilitando lo sviluppo e la distribuzione su diversi ambienti.

- **Docker Compose:** utilizzato per orchestrare i container Docker, semplificando la gestione dei servizi e delle dipendenze tra di essi.
 - Utilizzo di due compose file distinti, uno per l'ambiente di produzione ed uno per lo sviluppo, quest'ultimo provvisto di live reloading per facilitare lo sviluppo.
 - Implementazione di healthcheck per ogni servizio, in modo da verificare lo stato di ogni servizio e gestire al meglio la dipendenza tra i servizi (es. il backend dipende dal servizio meteo, quindi è stato configurato in modo che venga avviato solo dopo che il servizio meteo è risultato healthy).

5 Codice

5.1 Logica di Autorizzazione

Questo middleware rappresenta un componente fondamentale per la sicurezza delle API esposte. Tramite questo codice, il backend si assicura che solo gli utenti con privilegi di "Admin" possano eseguire determinate operazioni, come la manipolazione di regole e l'aggiunta di dispositivi. L'autenticazione viene gestita attraverso token JWT. Questo token viene qui estratto dall'header della richiesta HTTP, decodificato e verificato, permettendo al controller di procedere solo se l'utente risulta avere il ruolo di amministratore.

```
1 export const adminMiddleware = (  
2   req: Request,  
3   res: Response,  
4   next: NextFunction,  
5 ) => {  
6   const token = getTokenFromHeader(req);  
7   if (!token) {  
8     res.status(401).json({ error: "Unauthorized: Missing or invalid  
9       token" });  
10    return;  
11  }  
12  try {  
13    const decoded = verifyAuthToken(token) as JwpPayload;  
14    (req as any).user = decoded;  
15    if (decoded.role !== "Admin") {  
16      res.status(403).json({ error: "Forbidden: Admin access required"  
17        });  
18      return;  
19    }  
20    next();  
21  } catch (error) {  
22    res.status(401).json({ error: "Unauthorized: Invalid token" });  
23  }  
};
```

5.2 Validazione Input tramite Middleware

Questo blocco di codice illustra l'utilizzo della libreria `express-validator` per convalidare i parametri in ingresso alle API prima che vengano elaborati dai controller. Attraverso una catena di controlli dichiarativi, il middleware garantisce l'integrità dei dati: ad esempio, verifica che il parametro `temperature` sia un numero compreso tra 5 e 40 e che venga fornito solo se l'azione richiesta è effettivamente `setTemperature`. Questo approccio previene errori logici, iniezioni di dati malformati e disaccoppia la validazione sintattica dalla business logic.

```
1 export const temperatureValidator = [  
2   body("temperature")  
3     .if((_value, { req }) => req.params?.action === "setTemperature")  
4     .notEmpty()  
5     .withMessage("temperature is required when action is setTemperature  
6       ")  
7     .isNumeric()  
8     .withMessage("temperature must be a number")  
9 ]
```

```

8      .isFloat({ min: 5, max: 40 })
9      .withMessage("temperature must be between 5 and 40"),
10
11    body("temperature")
12      .if((_value, { req }) => req.params?.action !== "setTemperature")
13      .isEmpty()
14      .withMessage(
15        'temperature should only be provided when action is "
16          setTemperature"',
17      ),
18    validate,
19 ];

```

5.3 Notifiche Real-Time con Socket.io

Il codice qui presente viene utilizzato per inviare aggiornamenti di stato dei dispositivi in tempo reale ai client connessi. Quando lo stato di un sensore viene aggiornato nel backend, questo adapter utilizza Socket.io per emettere un evento specifico a tutti i client connessi della casa interessata, garantendo così che le interfacce utente riflettano immediatamente i cambiamenti senza necessità di refresh manuali. Il metodo `broadcast` centralizza la logica di invio dei messaggi, semplificando l'implementazione di notifiche per i vari tipi di eventi.

```

1  export class SocketIOSensorUpdateAdapter implements SensorUpdatePort {
2    constructor(
3      private io: Server,
4      private notificationPort?: HomeNotificationOutboundPort,
5    ) {}
6
7    private broadcast(homeId: string, event: string, payload: object):
8      void {
9      console.log(
10        'SocketIOSensorUpdateAdapter: Broadcasting ${event} to home-${
11          homeId}',
12      );
13      this.io.to('home-${homeId}').emit(event, payload);
14    }
15
16    sendInternalTemperatureUpdate(home: Home, update: TemperatureState):
17      void {
18      this.broadcast(home.id, "sensor:internal-temperature", {
19        temperature: update.temperature,
20      });
21    }
22  }

```

5.4 Integrazione con Modello AI (Streaming)

Questo snippet mostra come viene gestita la comunicazione con il modello AI in tempo reale. Invece di attendere la generazione completa della risposta, il metodo inoltra progressivamente i frammenti di testo generati al client tramite una porta dedicata (`ChatStreamPort`, che utilizza Socket.io). Prima di interrogare il modello, il conte-

sto viene costruito dinamicamente generando un system prompt che include le direttive specifiche per il modello AI.

```
1  async streamChat(  
2    homeId: string,  
3    username: string,  
4    userMessage: string,  
5    history: ChatMessage[],  
6    streamPort: ChatStreamPort,  
7    isAdmin: boolean,  
8  ): Promise<string> {  
9    if (!userMessage.trim()) {  
10      throw new Error("Message is required");  
11    }  
12  
13    const messages = await this.buildMessages(homeId, userMessage,  
14      history);  
15    return this.chatCompletionPort.streamChat(  
16      messages,  
17      this.options.model,  
18      homeId,  
19      streamPort,  
20      isAdmin,  
21    );  
22  }
```

6 Test

6.1 Euristiche di Nielsen

Per quanto riguarda la produzione di mockup e l'implementazione del frontend, è stato prestata particolare attenzione alle **Euristiche di Nielsen** per l'usabilità, al fine di garantire un'esperienza utente ottimale. Di seguito sono riportati alcuni esempi di come queste euristiche sono state applicate durante la progettazione dell'interfaccia utente:

- **Visibilità dello stato del sistema:** sono state implementate notifiche in tempo reale per informare gli utenti su eventi importanti. Sono stati inseriti feedback visivi immediati quando lo stato di un dispositivo smart viene modificato, ad esempio, un riscontro visuale sul toggle corrispondente.
- **Corrispondenza tra il sistema e il mondo reale:** sono stati utilizzati termini e icone familiari agli utenti per rappresentare i dispositivi smart e le azioni disponibili, ad esempio un'icona di una lampadina per rappresentare una lampadina smart.
- **Controllo e libertà dell'utente:** è stata implementata la possibilità di annullare azioni semplicemente, come il cambio istantaneo del toggle di dispositivi smart o la possibilità di eliminare facilmente regole di automazione. Inoltre, la navbar permette una navigazione semplice e veloce tra le diverse sezioni dell'applicazione.
- **Coerenza e standard:** è stata mantenuta una coerenza visiva e funzionale in tutte le pagine dell'applicazione, utilizzando la stessa palette colori, lo stesso stile dei pulsanti e lo stesso layout per elementi simili.
- **Prevenzione dell'errore:** abbiamo implementato meccanismi di prevenzione di errori, ad esempio, nel momento in cui l'utente agisce sul termostato: la temperatura viene modificata attraverso pulsanti di incremento/decremento che vengono disabilitati quando viene raggiunta la temperatura massima o minima consentita, evitando così l'inserimento di valori non validi.
- **Riconoscimento piuttosto che ricordo:** sono stati utilizzati menu a tendina, icone e pulsanti chiaramente etichettati per facilitare la navigazione e l'interazione, riducendo la necessità per gli utenti di ricordare comandi o percorsi specifici. Ad esempio, nella creazione di una regola di automazione i campi per la condizione e l'azione sono presentati come menu a tendina con opzioni predefinite, facilitando la selezione e riducendo la possibilità di errori, come l'inserimento di device non esistenti o azioni non valide.
- **Flessibilità e efficienza d'uso:** l'applicazione è stata progettata per essere utilizzata sia da utenti esperti che da utenti meno esperti. Ad esempio, una regola di automazione può essere creata tramite un'interfaccia guidata per utenti meno esperti, ma anche attraverso linguaggio naturale via chat, offrendo così un trade-off tra flessibilità e intuitività.
- **Estetica e design minimalista:** seguendo una filosofia "less is more", l'interfaccia è stata progettata per essere pulita, preferendo icone intuitive e layout semplici rispetto a elementi pesanti e ricchi di testo.

- **Aiutare gli utenti a riconoscere, diagnosticare e recuperare gli errori:** sono stati implementati messaggi di errore chiari e specifici, ad esempio, se un utente tenta di eseguire un'azione non consentita viene visualizzato un messaggio che spiega il motivo del blocco e suggerisce come risolvere il problema.
- **Aiuto e documentazione:** dando la priorità a un design intuitivo ed a un'interfaccia auto-esplicativa abbiamo ritenuto che l'applicazione non necessitasse di una sezione di aiuto dedicata.

6.2 Unit Testing e Strumenti di sviluppo

Per garantire il corretto funzionamento delle funzionalità principali, sono stati utilizzati test unitari per il backend. Questo permette di verificare frequentemente lo stato dell'applicazione e di avere sempre a disposizione test regressivi da effettuare ad ogni cambiamento o estensione delle funzionalità. Per i test unitari, sono stati utilizzati **Supertest** e **Vitest**. I test sono stati inseriti nella pipeline di Continuous Integration e nei Git Hook; perciò sono eseguiti automaticamente ad ogni push, ad ogni pull request e ad ogni commit sul branch principale, garantendo così che il codice presente in questi branch sia sempre coperto da test e che eventuali errori vengano individuati tempestivamente. Inoltre, i test ci hanno anche permesso di tenere sotto controllo la copertura del codice. Per assicurarsi la compatibilità cross-browser, l'applicazione è stata testata regolarmente su Google Chrome e Mozilla Firefox. Grazie alla possibilità di cambiare il tipo di device simulato all'interno degli strumenti di sviluppo, è stato possibile testare l'applicazione su una vasta gamma di dispositivi, dando la priorità ai dispositivi mobile, in ottica mobile-first.

6.3 Test di usabilità

Per validare il nostro design e la nostra progettazione, abbiamo svolto dei test di usabilità con utenti reali. Sono stati individuati cinque task e tre metriche principali per valutare l'esperienza utente.

6.3.1 Metriche

- **Reattività:** giudizio assegnato dagli utenti al termine di ogni task (da 1 a 5), in base alla reattività del sistema.
- **Facilità percepita:** giudizio che va da 1 (molto difficile da usare) a 5 (molto facile da usare), assegnato dagli utenti per valutare la facilità d'uso dell'applicazione.
- **Estetica:** giudizio che va da 1 (poco piacevole) a 5 (molto piacevole), assegnato dagli utenti per valutare l'estetica dell'interfaccia.

6.3.2 Elenco dei task

- **Aggiunta dispositivo:** l'utente (Admin) deve aggiungere una nuova lampadina smart in salotto. Successivamente, deve accenderla e spegnerla tramite l'app.
- **Verifica previsioni meteo:** l'utente deve chiedere al chatbot le previsioni meteo per la settimana successiva.

- **Creazione automazione:** l'utente (Admin) deve creare una regola di automazione che preveda l'apertura della finestra in soggiorno quando la temperatura interna supera i 25 gradi.
- **Verifica interazioni su dispositivo:** l'utente (Admin) deve verificare chi ha spento per ultimo la luce in cucina.
- **Programmazione giornaliera:** l'utente deve modificare la programmazione giornaliera, impostando a 17 gradi la temperatura tra le 11 e le 12.
- **Storico qualità dell'aria:** l'utente deve visualizzare l'andamento della qualità dell'aria durante la settimana precedente.

6.3.3 Dati rilevati

Reattività					
Task	Riccardo	Nicolas	Matteo	Agostino	Livia
Aggiunta dispositivo	5	5	5	5	5
Verifica previsioni meteo	3	4	4	4	5
Creazione automazione	4	5	4	5	4
Verifica interazioni su dispositivo	5	5	5	5	5
Programmazione giornaliera	5	5	4	5	5
Storico qualità dell'aria	5	5	5	5	5

Tabella 1: Valutazione degli utenti sulla Reattività

Facilità percepita					
Task	Riccardo	Nicolas	Matteo	Agostino	Livia
Aggiunta dispositivo	5	4	5	4	5
Verifica previsioni meteo	4	5	4	5	5
Creazione automazione	4	5	5	5	4
Verifica interazioni su dispositivo	5	5	4	5	5
Programmazione giornaliera	4	5	5	4	5
Storico qualità dell'aria	5	5	5	5	5

Tabella 2: Valutazione degli utenti sulla Facilità percepita

Estetica					
Task	Riccardo	Nicolas	Matteo	Agostino	Livia
Aggiunta dispositivo	5	5	5	5	5
Verifica previsioni meteo	4	4	5	5	4
Creazione automazione	5	5	4	5	5
Verifica interazioni su dispositivo	5	5	5	5	5
Programmazione giornaliera	5	4	5	5	5
Storico qualità dell'aria	5	5	5	5	5

Tabella 3: Valutazione degli utenti sull'Estetica

Come si può vedere dalle tabelle, l'applicazione è risultata molto apprezzata dagli utenti, con giudizi molto positivi su tutte le metriche. I test hanno sollevato qualche piccolo problema di usabilità, principalmente riguardante l'implementazione della chat, che ci sono serviti come spunto di discussione e che sono stati poi risolti.

In particolare:

- Alcuni utenti hanno segnalato che, nel momento in cui viene inviato un messaggio, il tempo di attesa è troppo lungo prima di vedere un feedback dal modello AI.
- E' stata segnalata la mancanza di un messaggio di benvenuto nella chat che fosse in grado di guidare l'utente su come interagire con il modello AI e su quali tipi di domande fosse possibile fare.
- Alcuni utenti hanno evidenziato come l'interazione del chatbot con il sistema di HiHome potesse essere resa più trasparente, ad esempio mostrando le azioni che il modello AI sta eseguendo in tempo reale, in modo da rendere più chiaro il collegamento tra le risposte del modello e le azioni che vengono eseguite sui dispositivi smart.

Per fare fronte a questi problemi abbiamo implementato le seguenti modifiche:

- E' stato introdotto lo streaming delle risposte del modello AI, utilizzando la connessione websocket, in modo da restituire all'utente un feedback continuo. Inoltre, nel momento in cui la risposta viene processata, è stato inserito un placeholder di caricamento ("Thinking...").
- E' stato aggiunto un messaggio di benvenuto nella chat che guida l'utente su come interagire con il modello AI e su quali tipi di domande sia possibile fare.
- Ogni volta che un tool viene chiamato appare sulla chat (attraverso la connessione websocket) il nome del tool stesso, per dare trasparenza all'interazione del modello AI con il sistema di HiHome.

7 Deployment

Il sistema HiHome può essere facilmente distribuito ed eseguito localmente sia in modalità di "produzione" che di sviluppo (con hot-reloading abilitato per consentire lo sviluppo attivo) grazie all'utilizzo di Docker Compose.

7.1 Comandi per il Deployment

Di seguito sono descritti i comandi principali per gestire il ciclo di vita dell'applicazione localmente.

7.1.1 Esecuzione Container (Produzione)

Per compilare ed avviare l'intero stack in modalità di produzione:

```
1 # Avvio di tutti i container
2 docker compose up
3
4 # Arresto dei container con rimozione dei volumi associati (es.
   database)
5 docker compose down -v
```

Inoltre, è possibile scaricare direttamente le immagini dei container dalla sezione "Releases" del repository GitHub, poiché la pipeline di CI è configurata per effettuare il push delle immagini Docker ad ogni commit sul main.

7.1.2 Esecuzione con Hot Reloading

Per lo sviluppo, è presente una configurazione dedicata che monta i file locali come volumi all'interno dei container, consentendo il live reloading del codice ad ogni modifica effettuata.

```
1 # Avvio dei container in dev mode
2 docker compose -f docker-compose.dev.yml up
3
4 # Arresto dei container in dev mode
5 docker compose -f docker-compose.dev.yml down -v
```

7.1.3 Esecuzione dei Test

Per verificare la correttezza del codice ed eseguire gli unit test:

```
1 # Esecuzione unit test per il backend Node.js
2 pnpm run test
3
4 # coverage per il backend Node.js
5 pnpm run test:coverage
6
7 # Esecuzione dei test per il servizio Go
8 go test -C ext-api-service -v ./...
9
10 # Coverage per il servizio Go
11 go test -C ext-api-service -coverprofile=coverage.out -v ./...
```

7.2 Endpoint e Indirizzi Locali

Una volta che l'applicazione è in esecuzione localmente, i vari moduli ed endpoint esposti possono essere interrogati localmente tramite i seguenti indirizzi web:

- **Frontend Application (Vue 3):** Raggiungibile all'indirizzo `http://localhost:8080`.
Costituisce l'interfaccia utente principale per l'interazione con la smart home.
- **Backend REST API (Node.js/Express):** Raggiungibile all'indirizzo `http://localhost:3000`.
Ad esempio, è possibile testare lo stato di salute del backend effettuando una richiesta GET su `http://localhost:3000/api/health`.
- **External API Service (Go):** Raggiungibile esternamente all'indirizzo `http://localhost:8081`.
L'endpoint di health check per verificare lo stato di questo modulo è accessibile a `http://localhost:8081/health`.

8 Conclusioni

Il gruppo di lavoro ritiene di aver raggiunto con successo tutti i traguardi prefissati in fase di analisi. I test di usabilità condotti sul campo hanno confermato che l'interfaccia utente, sviluppata seguendo pattern moderni e con un approccio mobile-first, risulta fin da subito chiara ed estremamente accessibile anche per utenti non tecnici. I test unitari hanno supportato la stabilità del sistema, garantendo che le funzionalità principali funzionassero correttamente e che eventuali regressioni venissero individuate tempestivamente.

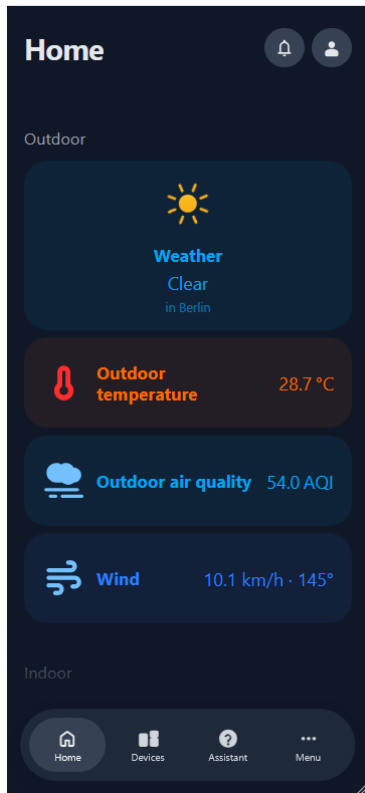
Seppur le interazioni con i dispositivi siano simulate, in quanto abbiamo ritenuto che la parte IoT non fosse il focus del progetto, l'applicazione è stata progettata per gestire in maniera efficace i casi d'uso proposti e per essere facilmente estendibile. Durante lo svolgimento del progetto abbiamo avuto la possibilità di diventare più pratici con lo stack MEVN e lo sviluppo di applicazioni web. Particolarmente interessante è stata l'integrazione con servizi esterni, come il modello AI ed il servizio per il meteo, oltre che l'implementazione della comunicazione tra i vari servizi del sistema, caratterizzata da interazioni ed aggiornamenti real-time.

Il gruppo ha lavorato in modo coeso e collaborativo, suddividendo equamente i compiti. La complessità del dominio ha fornito più volte spunti di riflessione, che sono stati affrontati in maniera costruttiva portando a soluzioni condivise e discussioni interessanti.

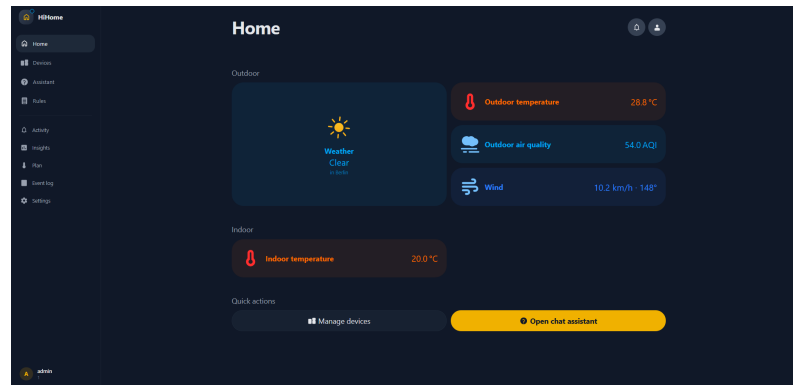
Gli sviluppi futuri includono l'estensione dei dispositivi supportati e l'integrazione con dispositivi reali, ad esempio utilizzando servizi di terze parti o API fornite dai produttori.

9 Screenshots

Sono di seguito allegati gli screenshot finali dell'applicazione, sia in versione mobile che in versione desktop.

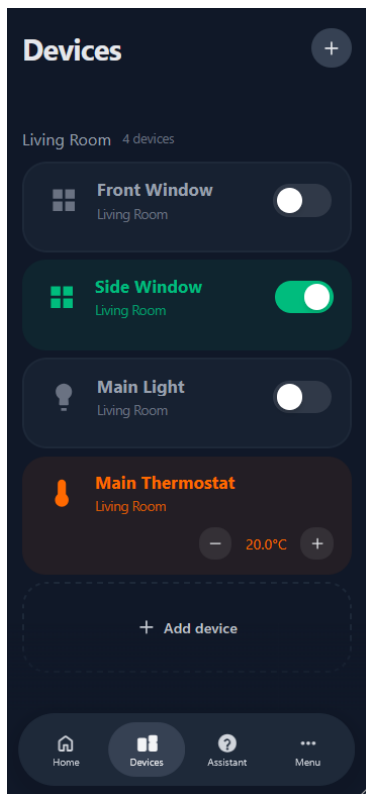


(a) Mobile

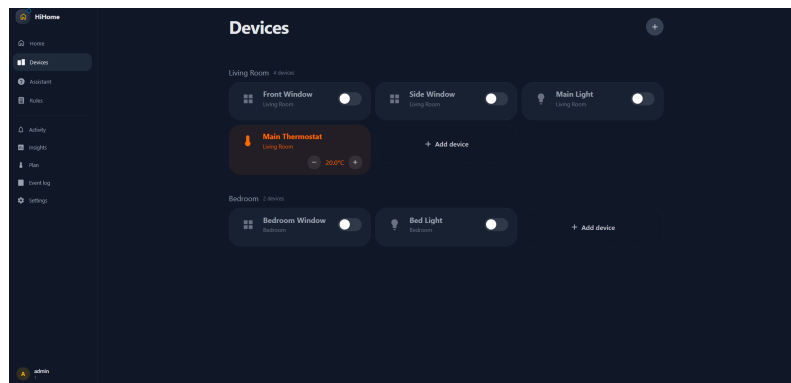


(b) Desktop

Figura 14: Dashboard

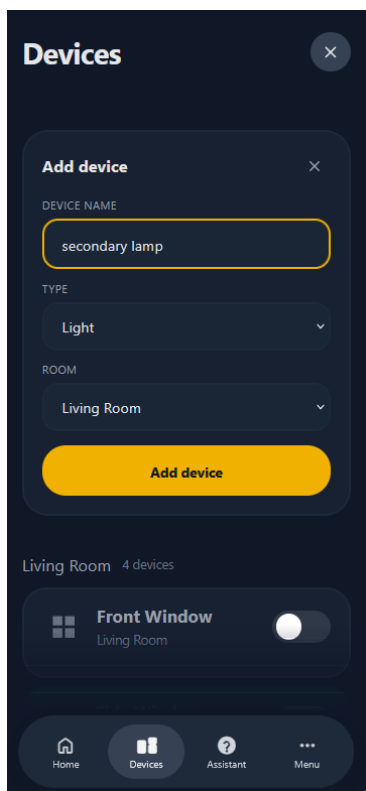


(a) Mobile

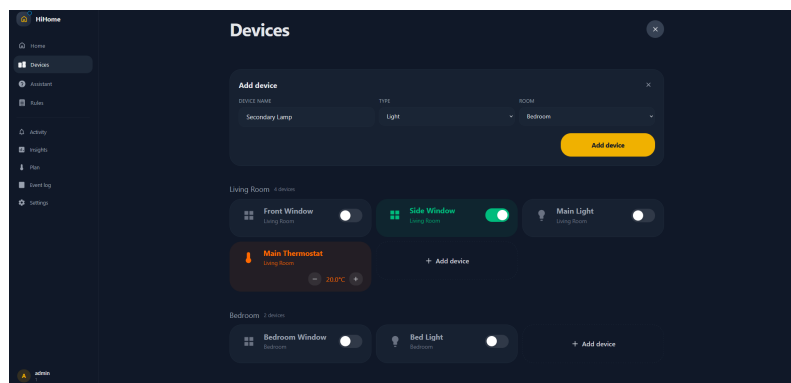


(b) Desktop

Figura 15: Lista device

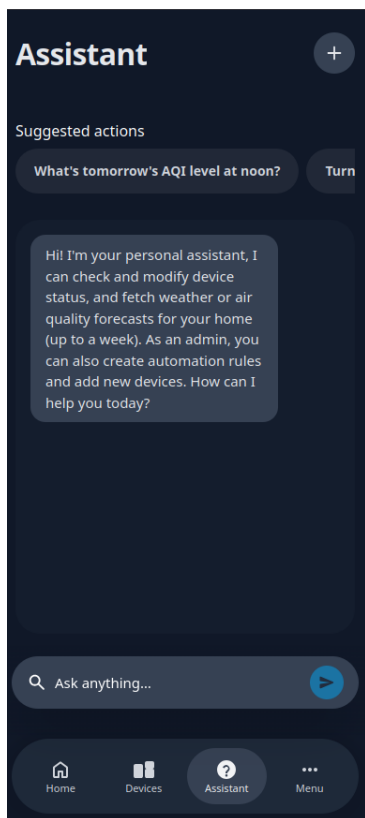


(a) Mobile

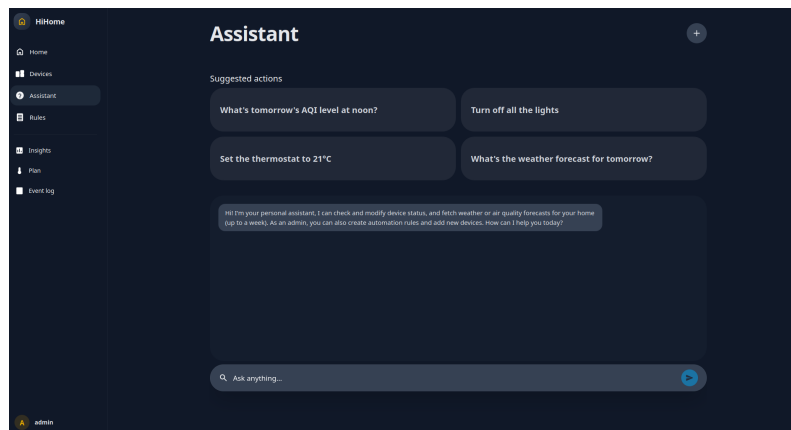


(b) Desktop

Figura 16: Aggiunta di un nuovo device

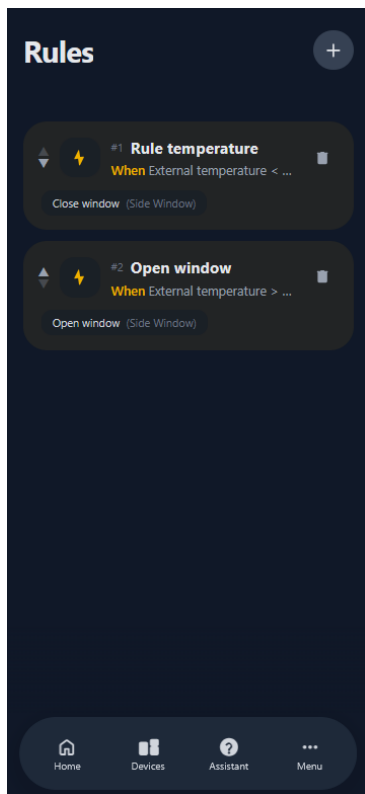


(a) Mobile

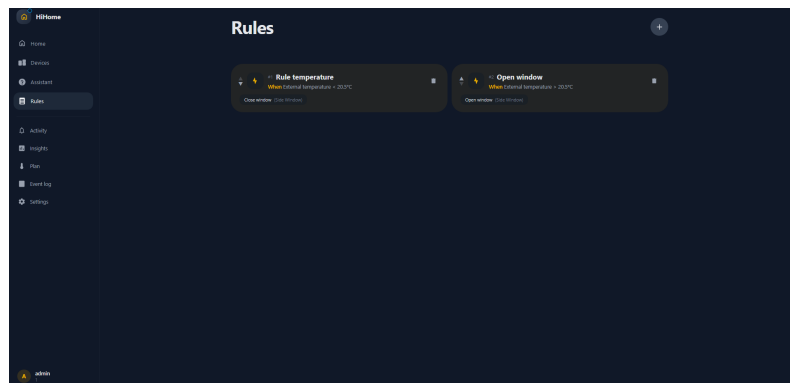


(b) Desktop

Figura 17: Chatbot

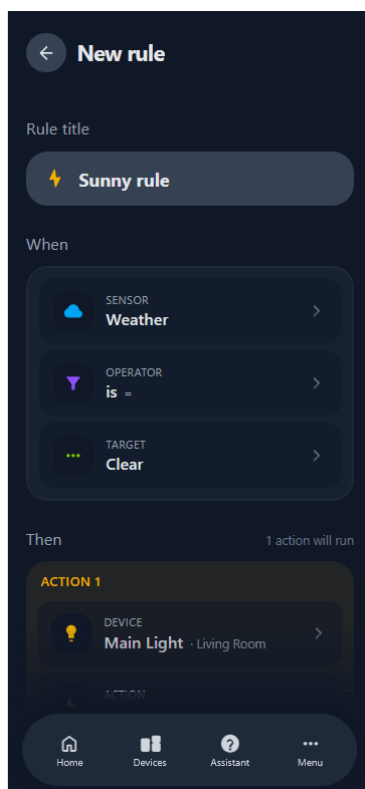


(a) Mobile

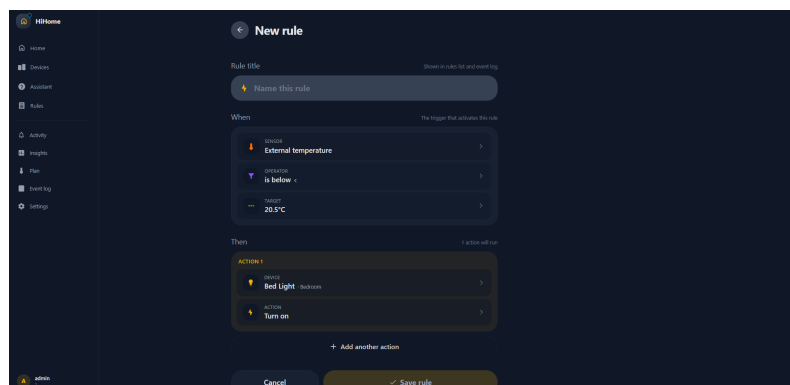


(b) Desktop

Figura 18: Regole di automazione

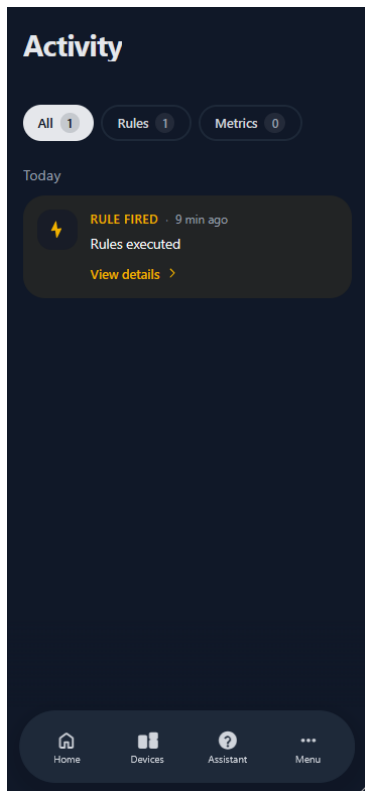


(a) Mobile

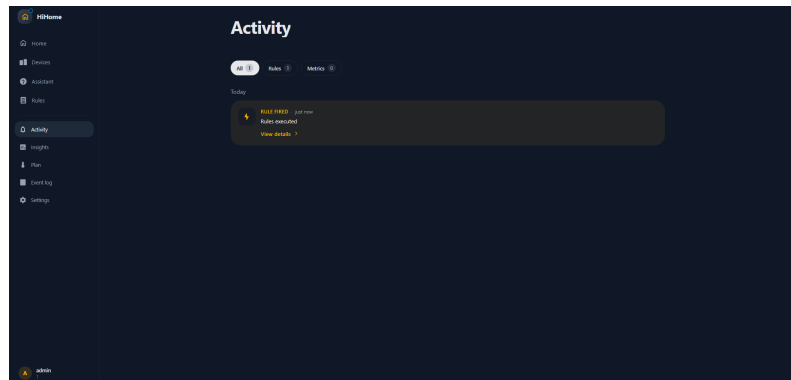


(b) Desktop

Figura 19: Creazione nuova regola di automazione

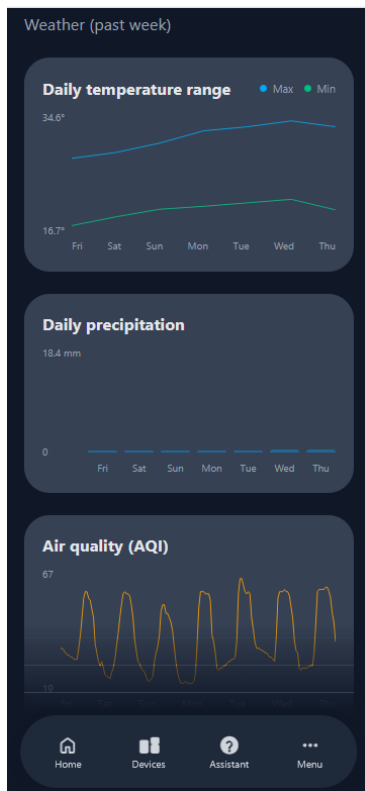


(a) Mobile



(b) Desktop

Figura 20: Notifiche

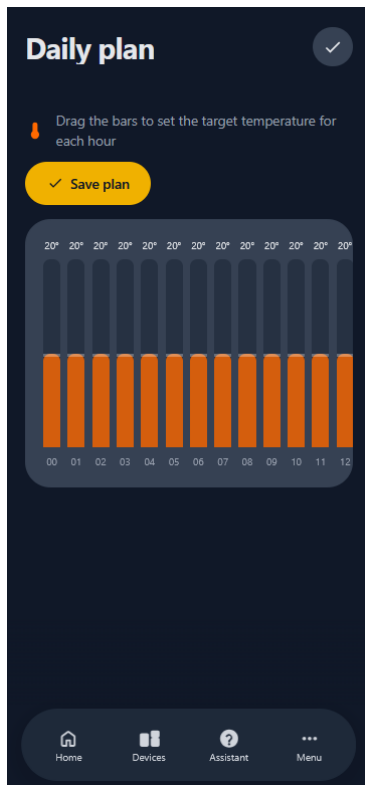


(a) Mobile

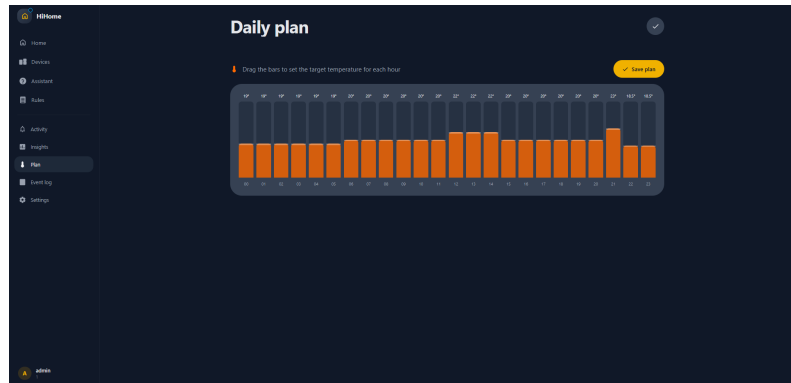


(b) Desktop

Figura 21: Report settimanale

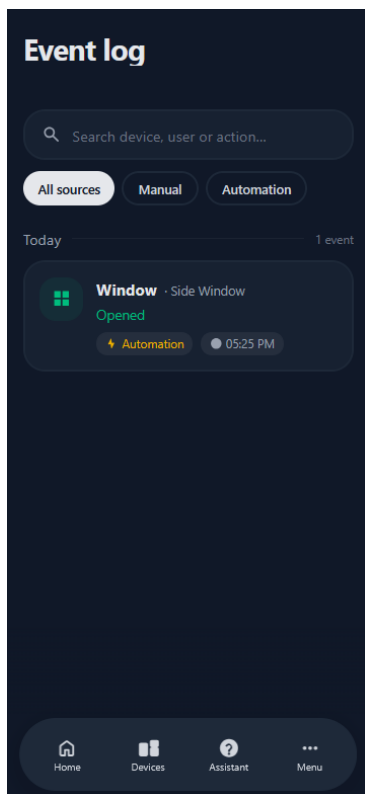


(a) Mobile

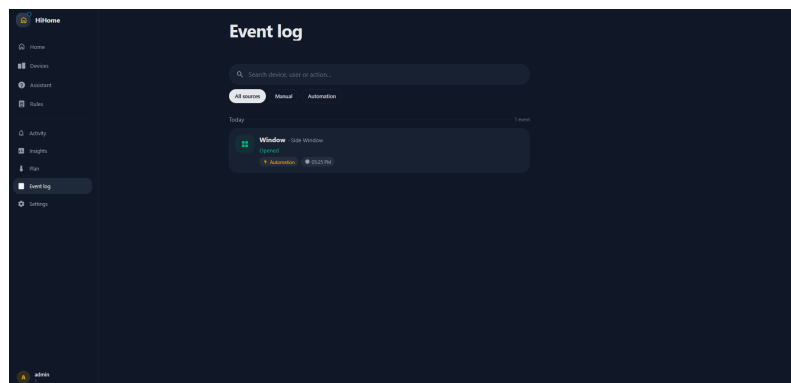


(b) Desktop

Figura 22: Programmazione settimanale del termostato

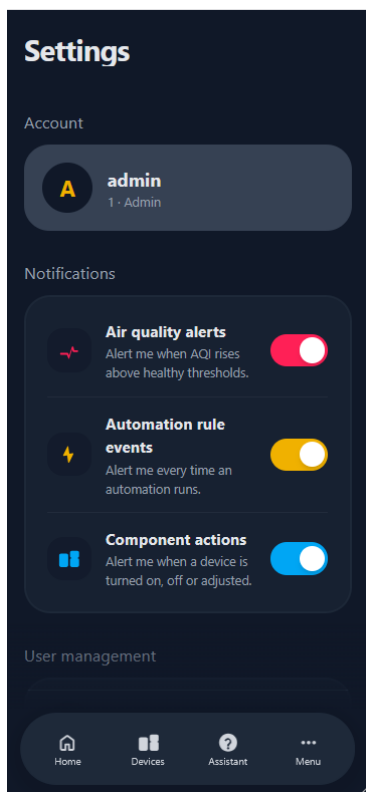


(a) Mobile

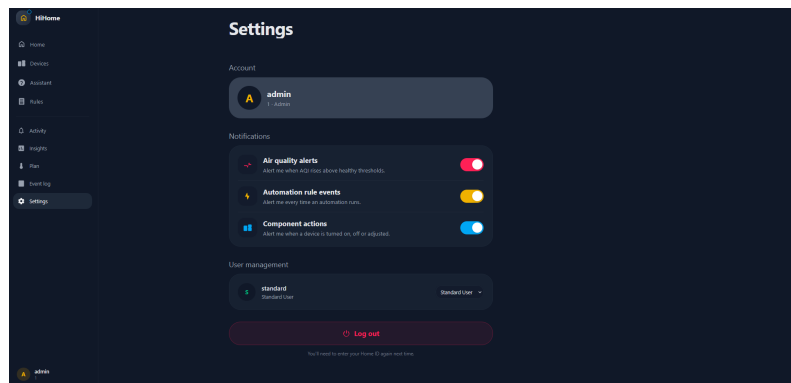


(b) Desktop

Figura 23: Log di eventi



(a) Mobile



(b) Desktop

Figura 24: Impostazioni